QR Code Scanner Security: A Black-Box Testing Study

Jedidiah Akinola
joa24@sfu.ca

Annie Boltwood
aeb17@sfu.ca

Mahim Chaudhary
mca265@sfu.ca

Simon Fraser University
CMPT 479: Special Topics in Computing Systems
April 12, 2026

Abstract

QR codes have become a routine part of mobile computing, appearing in payments, menus, app onboarding, device setup, login flows, support portals, and public signage. That convenience also creates a security problem. The content of a QR code is not human-readable before scanning, so users rely on the scanner application to decode, classify, preview, and hand off the payload safely. In effect, QR scanner apps sit at a decision point between an opaque visual token and a potentially security-sensitive action such as opening a browser, launching a dialer, or passing data to another app.

This paper presents a black-box study of three Android QR scanner applications: Gamma Play, Binary Eye, and QR Scanner Plus. We evaluated them using a controlled corpus of 12 benign QR payloads spanning phishing-style URLs, deceptive formatting, shortened links, oversized payloads, and nonstandard URI schemes. Across 36 total tests, we measured preview clarity, confirmation behavior, handoff behavior, and parsing robustness. Our results show that all three apps ultimately handled the tested payloads safely in the sense that none auto-opened content and no visible unsafe execution was observed. However, the apps differed substantially in preview clarity and decoding robustness. Gamma Play and QR Scanner Plus consistently exposed full destinations before handoff, while Binary Eye generally exposed only partial destination visibility and failed to decode several oversized and `javascript:` payloads. None of the three apps displayed heuristic or cautionary warnings for suspicious-looking but benign payloads. The central finding of the study is that QR scanner security is not only about malicious-domain detection. It is also about the tradeoff between preview clarity, parser robustness, and how much interpretation happens inside the scanner before control is handed to another app.

1 Introduction

QR codes are now ordinary infrastructure. They are used to open restaurant menus, pair devices, authenticate sessions, initiate payments, redeem coupons, launch support flows, and route users into web content. DENSO WAVE, the inventor of the QR Code, notes that the technology was standardized internationally and intentionally designed for broad adoption and high-capacity encoding [2, 1]. That success creates a familiar security pattern. A technology optimized for speed and convenience often reduces the amount of context a user sees before acting.

Phishing remains one of the most effective attack families because it uses social engineering to convert routine action into compromise. CISA and NIST both emphasize that phishing succeeds by convincing users to follow links, visit spoofed destinations, or provide sensitive information under the guise of legitimacy and urgency [3, 4]. QR codes make this problem harder to evaluate at a

glance because the user cannot inspect the underlying destination until after scan time. A domain that might look suspicious in plain text becomes visually opaque inside a matrix barcode.

This matters especially on mobile devices. Mobile interfaces already compress or hide security cues, and QR scanning adds one more abstraction layer between the user and the destination. In Android, the scanner application often decodes a payload and then triggers another component by means of an implicit intent, such as opening a browser, dialer, or search flow [5, 6]. That means scanner apps are not passive decoders. They make real security-relevant decisions about how content is presented and where it is routed.

Prior work and public guidance suggest that QR-related attacks are not hypothetical edge cases. QR code phishing, often called quishing, is now widely recognized as a real threat pattern in public guidance from government and industry sources [10, 11]. Recent academic work strengthens that concern. Kowalewski et al. found that users were far less likely to recognize QR-mediated scams than comparable non-QR attacks, suggesting that the QR format itself obscures risk [8]. Zhang et al. also show that QR-based login and related QR-mediated workflows can create real security challenges in deployed systems [9]. Together, these results suggest that the scanner layer deserves closer study.

This paper asks a focused and practical question: when common Android QR scanner applications are given suspicious-looking, unusual, or unusually large payloads, what do they actually do? We answer this through a controlled black-box study of three scanner apps and a 12-payload corpus designed to probe preview quality, confirmation behavior, handoff design, and parser robustness without exposing testers to live malicious infrastructure.

2 Background and Related Work

2.1 Why QR Codes Create a Distinct Security Problem

A QR code can encode ordinary URLs, phone numbers, free text, and other data forms [2, 1]. That flexibility is useful, but it means the scanner must interpret not only the bits but also the intended type of action. On Android, implicit intents allow an application to request an external action based on a URI or data type, which is precisely the mechanism many QR apps rely on after decode [5, 6]. A scanner can therefore do several things after reading a code:

1. show the content inside the app,
2. ask the user for confirmation,
3. route the content externally to another app, or
4. fail to decode or classify the payload.

From a security perspective, these are not equivalent outcomes. A full preview followed by explicit confirmation gives the user a meaningful chance to inspect the destination. A brief or partial preview gives much less opportunity. A decode failure may avoid dangerous execution, but it also indicates weaker robustness. An external handoff may be safe, but it may reduce the scanner's ability to contextualize the content before control leaves the app.

2.2 Related Work on QR Scanner Security

Wahsheh and Luccio conducted a broad study of 100 barcode scanner applications and found substantial variation in security and privacy properties across apps [7]. Their work is particularly

relevant because it shows that scanner applications differ not only in permissions and privacy posture, but also in user protection features such as URL checking. Our study complements that work by narrowing the focus to user-visible black-box behavior under a controlled corpus.

2.3 Related Work on User Susceptibility and QR Attacks

More recent work highlights the user side of the problem. Kowalewski et al. studied QR-based attacks in realistic tasks and found that users who could detect traditional phishing often failed to identify QR-mediated variants [8]. In one of their reported comparisons, participants recognized fraudulent QR payment requests at much lower rates than equivalent manually entered payment details. This suggests that a scanner's preview and confirmation design may matter even more in QR-mediated workflows than in ordinary browsing.

Additional public guidance from the American Hospital Association's HC3 report and the United States Postal Inspection Service also treats quishing as an emerging operational threat, especially because QR codes can be embedded into physical environments where users assume legitimacy [10, 11]. These sources are not substitutes for academic evidence, but they help illustrate why the topic matters in practice.

2.4 Positioning Our Contribution

Our study does not attempt to measure whether apps can detect known malicious infrastructure through threat intelligence feeds or domain reputation services. Instead, it focuses on a narrower and still important question: *how scanner apps behave when the payload itself looks suspicious, unusual, or large.* This makes the study ethically safer, reproducible, and directly aligned with the observable user experience.

3 Research Question and Study Design

3.1 Research Question

Our central research question was:

> How do common Android QR scanner applications handle suspicious-looking or unusual QR payloads, especially with respect to preview clarity, confirmation behavior, handoff behavior, and parsing robustness?

This question was intentionally framed around *handling behavior* rather than around live malicious-domain detection. A scanner app can still be more or less security-conscious even when none of the tested domains are actually malicious.

3.2 Scope

The study examines:

- how much of a decoded destination is shown before user action,
- whether the app requires confirmation before handoff,
- where the content is handed off,
- whether unusual or large payloads decode correctly,

- whether the app displays any cautionary or warning UI.

The study does **not** claim to measure:

- live malicious-domain detection,
- anti-phishing intelligence integration,
- network telemetry behavior,
- hidden internal processing beyond what is visible in the interface.

4 Methodology

4.1 Target Applications

We selected three Android QR scanner applications:

1. **Gamma Play**
2. **Binary Eye**
3. **QR Scanner Plus**

These applications were chosen because they were installable and testable in our emulator environment, represented visibly different interface styles, and were realistic choices for a small comparative study. Our goal was not to identify a universally best scanner, but to compare three representative designs under the same conditions.

4.2 Test Environment

All tests were conducted in an Android emulator using manual interaction. Each payload was encoded as a QR image and then loaded through the scanner application's image-import or gallery-based flow. This approach helped standardize the input path across apps and reduced variability that might arise from camera focus, lighting, or live-scanning conditions.

For each test, we recorded:

- the scanner's result or preview screen,
- the post-handoff screen when relevant,
- a structured row in a results CSV,
- notes describing app behavior.

The final dataset was audited after testing and verified to contain exactly 36 completed rows, with no missing combinations, duplicate rows, malformed values, or broken screenshot references.

4.3 Payload Corpus

We constructed a corpus of 12 benign payloads across five categories. Each category was designed to probe a different part of scanner behavior.

Table 1: Payload categories used in the study

Category	Rationale	Count
Phishing-style URLs	Benign links with common phishing framing such as login, verify-account, or billing language	3
Deceptive formatting	Benign links with visually suspicious brand-adjacent wording or impersonation-style structure	2
Shortened links	Benign short-path or redirect-like links that reduce immediate interpretability	2
Oversized payloads	Very long URL and long plain-text payloads used to probe parser robustness and preview behavior	2
Nonstandard schemes	`tel:`, `data:`, and `javascript:` payloads used to observe classification and handoff choices	3

Representative examples included:

- `https://example.com/login`
- `https://example.com/paypa1-helpdesk/login`
- `https://example.org/out?target=acct-reset`
- `tel:+16045551234`
- `data:text/plain,harmless-qr-study-payload`
- `javascript:console.log('qr-study-harmless-string')`

The domains `example.com` and `example.org` were intentionally used as safe placeholders. This allowed us to test suspicious presentation without directing devices to live malicious content.

4.4 Recorded Variables

For each test, we recorded the following fields:

1. target application,
2. payload identifier,
3. payload category,
4. whether the full destination was visible,
5. whether a warning was shown,
6. whether confirmation was required,
7. whether the app auto-opened content,

8. handoff type,
9. whether the payload was handled safely,
10. notes and screenshot references.

4.5 Definition of Safe Handling

In this study, a payload was treated as *handled safely* if the app did not automatically execute or launch the action without user input and if no visible unsafe execution was observed. This is a deliberately narrow definition based on black-box evidence. It does not imply that the app is secure in all respects.

5 Results

5.1 Dataset Overview

The final dataset contains 36 completed rows: 12 tests for Gamma Play, 12 for Binary Eye, and 12 for QR Scanner Plus. Coverage was complete across all expected target-payload combinations. The post-test audit found:

- no missing rows,
- no duplicate target-payload pairs,
- no inconsistent target names,
- no inconsistent payload identifiers,
- no malformed CSV values,
- no missing screenshots referenced by the CSV.

5.2 Application-Level Summary

Table 2 provides a concise per-application summary.

Table 2: Per-application summary of key outcomes

App	Full Yes	Partial	None	Decode Fail	Warnings	Auto-open
Gamma Play	12	0	0	0	0	0
Binary Eye	0	9	3	3	0	0
QR Scanner Plus	12	0	0	0	0	0

Two high-level findings are immediately clear. First, none of the 36 tests resulted in automatic opening or visible automatic execution. Second, none of the three apps displayed a warning for any tested payload. The interpretation of the second result requires care. Since the test corpus used benign placeholder domains, we do not treat the absence of warnings as evidence that the apps failed to block known malicious infrastructure. Instead, we interpret it more narrowly: in this dataset, the apps did not present heuristic or cautionary warnings for suspicious-looking but benign content.

5.3 Gamma Play

Gamma Play was the most differentiated of the three applications. In all 12 tests, it showed a persistent in-app result screen with full destination visibility before handoff. It also consistently required explicit user action and never auto-opened content.

The app differentiated among payload types more explicitly than the others:

- URL-like payloads opened in a Chrome-backed in-app browsing flow.
- The `tel:` payload led to a dialer handoff without placing the call automatically.
- The `data:` payload was shown through an internal text-viewer-like screen.
- The `javascript:` payload was exposed and routed into a viewer-like screen, with no visible execution.
- Both oversized payloads decoded successfully.

Gamma Play therefore offered the richest scanner-side interpretation. It kept the user inside the scanning context longer and made the payload class more apparent before handoff.

5.4 Binary Eye

Binary Eye showed the leanest interface and the weakest parser in our dataset. When it successfully decoded URL-like content, it generally presented only a brief transient preview or action chip before handing off to Chrome. That behavior produced partial destination visibility in nine successful rows rather than full visibility.

Binary Eye also failed to decode three cases:

- `SCHEME_003` (`javascript:`)
- `OVER_001` (oversized URL)
- `OVER_002` (oversized text)

In each of these cases, the app displayed "No barcode found" rather than exposing or executing the content. From a safety perspective, this failure mode avoided obvious dangerous behavior. From a robustness perspective, however, it reveals weaker parsing and less informative feedback.

Binary Eye's design appears to prioritize speed and minimalism. For some users that may be attractive, but in our dataset the cost was reduced preview clarity and lower robustness on unusual or large inputs.

5.5 QR Scanner Plus

QR Scanner Plus looked closer to Gamma Play than to Binary Eye in its preview model. In all 12 tests, it showed a persistent result page with full payload visibility before handoff. It also required user action and did not auto-open content.

Its handling logic was somewhat more uniform than Gamma Play's:

- ordinary links opened externally in Chrome,
- `tel:` opened the dialer with the number prefilled,

- `data:`, `javascript:`, and oversized plain-text content were shown clearly on the result page and then routed into web search rather than an internal viewer.

This means QR Scanner Plus combined strong preview clarity with a simpler and more externalized handoff model. It exposed the user to more context than Binary Eye, but it performed less scanner-side interpretation than Gamma Play for nonstandard or text-like payloads.

5.6 Cross-App Comparison

Table 3 summarizes the strongest behavioral differences.

Table 3: Behavioral comparison across applications

Dimension	**Gamma Play**	**Binary Eye**	**QR Scanner Plus**
Preview clarity	Persistent in-app preview, full visibility	Brief transient preview, usually partial visibility	Persistent result page, full visibility
Handoff style	Richer and more type-specific	Fast external handoff when decode succeeds	Clear preview, then mostly external Chrome or search
Oversized payloads	Decoded both oversized cases	Failed both oversized cases	Decoded both oversized cases
`data:` handling	Internal text-viewer-like handling	Search/browser handoff	Search/browser handoff
`javascript:` handling	Exposed but not visibly executed	Failed to decode	Exposed, then routed to search
Warnings	None observed	None observed	None observed

Several themes emerge from this comparison.

Preview clarity. Gamma Play and QR Scanner Plus consistently gave the user a full preview before handoff. Binary Eye did not. This may be the most visible difference in the entire study.

Parsing robustness. Gamma Play and QR Scanner Plus successfully decoded all 12 payloads each. Binary Eye failed on three cases, all of which involved unusual structure or unusually large size.

Interpretation strategy. Gamma Play interpreted more inside the app. QR Scanner Plus exposed content clearly but tended to externalize the next step. Binary Eye minimized scanner-side presentation the most.

6 Discussion

The most important takeaway from this study is that QR scanner security should not be framed only as a yes-or-no question about malicious URL detection. A real user experiences scanner security through interface design and content handling. What matters in practice is whether the app shows enough of the destination, whether it asks the user to confirm before acting, whether it classifies unusual content sensibly, and whether it remains robust when the input becomes large or awkward.

Seen from that perspective, the three applications occupy different positions.

Gamma Play appears to favor scanner-side interpretation. It preserves context, clearly differentiates among payload types, and lets the user inspect content before handoff. That can be beneficial because users need context in order to reject suspicious requests. The tradeoff is that the scanner app itself takes on more responsibility for content interpretation. In our tests, that did not produce visible unsafe behavior, but it is still a design choice worth noting.

Binary Eye favors minimalism. It keeps the interface light and moves quickly to Chrome or another handler. This may reduce interface clutter, but our results suggest two security costs. First, the user gets less persistent opportunity to evaluate the destination. Second, parser failures on the more unusual cases indicate reduced robustness. Minimalism is not necessarily wrong, but it is not free.

QR Scanner Plus occupies a middle position. It offers strong preview clarity and explicit confirmation, which are both positive. At the same time, it externalizes interpretation more than Gamma Play does, especially for `data:`, `javascript:`, and large text payloads, which it routes to search instead of resolving internally.

6.1 Interpreting the Absence of Warnings

One point deserves special care. None of the three apps displayed a warning. This should *not* be interpreted to mean that all three apps failed to detect malicious sites in the usual blacklist or reputation sense. Our corpus used `example.com` and `example.org`, which are safe placeholder domains. A reputation system would have little reason to block them.

A more accurate reading is this: none of the apps offered any visible cautionary labeling for suspicious-looking but benign payloads. For example, the apps did not highlight deceptive brand-adjacent formatting, shortened-style links, or unusual schemes as potentially risky classes. Whether a scanner *should* do that is partly a design question, but it is still a meaningful observation because public phishing guidance often emphasizes careful destination inspection and skepticism toward urgent or brand-impersonating requests [3, 4, 10]. If scanners are going to rely on users to make those judgments, then preview clarity becomes even more important.

6.2 What the Study Suggests About User Protection

Taken together, the data suggests that scanner apps provide user protection in three distinct ways:

1. by exposing the payload clearly,
2. by avoiding automatic action,
3. by choosing conservative handoff behavior for unusual inputs.

All three apps succeeded on the second point. They did not auto-open content or visibly auto-execute the tested payloads. The more consequential differences were on the first and third points. Gamma Play and QR Scanner Plus gave clearer previews than Binary Eye. Gamma Play also handled unusual schemes more explicitly inside the app, while QR Scanner Plus preferred clearer preview followed by external search or browser handoff. These are different safety philosophies.

7 Limitations and Threats to Validity

This study has several limitations.

First, it is a black-box study of only three Android scanner applications. The results should not be generalized to all QR scanners, all Android versions, or iOS devices.

Second, the study used safe placeholder domains and benign simulated payloads. This was an ethical and methodological choice. It allowed us to study scanner behavior without exposing devices or user accounts to real phishing sites. However, it also means the paper does not measure live threat-intelligence blocking or real malicious-domain detection.

Third, the testing process was manual. Although the dataset was carefully audited after completion, black-box observation still depends on what can be seen in the interface. We cannot claim visibility into internal parsing libraries, network activity, analytics, or hidden security checks.

Fourth, emulator-based testing may differ from physical-device testing in small ways, especially around app integration or system defaults. Even so, the observed outcomes in this study, such as preview visibility, dialer handoff, browser handoff, search routing, and decode failure, are directly relevant to user experience and remain useful comparative signals.

Finally, the corpus was intentionally small and focused. A larger study could include additional scanner apps, more scheme types, malicious-domain simulations with a local test server, network instrumentation, or repeated trials across devices.

8 Artifacts and Reproducibility

The study artifacts are available in the project repository [12]. The repository includes:

1. the QR corpus,
2. payload manifests,
3. the final results CSV,
4. screenshots for all completed tests,
5. project notes and supporting material.

Repository:

https://github.com/annieboltwood/QR-Code-Security-CMPT479

These artifacts make the study reproducible at the level of input payloads, observed outcomes, and result structure. They also satisfy the assignment requirement for publicly available artifacts with documentation.

9 Conclusion

This paper presented a 36-test black-box study of three Android QR scanner applications using a controlled corpus of suspicious-looking, unusual, and oversized QR payloads. The central conclusion is that QR scanner security is best understood as a tradeoff among preview clarity, parser robustness, and handoff design.

Gamma Play offered the most differentiated handling and the richest scanner-side interpretation. QR Scanner Plus matched Gamma Play on preview clarity but relied more heavily on external Chrome and web search for some payload classes. Binary Eye was the most minimal, but also the least robust, with partial destination visibility in most successful URL-like cases and outright decode failures on several unusual inputs.

All three apps avoided automatic opening and no visible unsafe execution was observed. That is an encouraging result. At the same time, none of the apps displayed heuristic cautionary warnings for suspicious-looking but benign payloads. In practice, this means users still depend heavily on what the scanner chooses to show before handoff. When the code itself is unreadable until scanned, the scanner application becomes part of the trust model. The clearer and more robust that scanner is, the better chance the user has to make a safe decision.

References

[1] DENSO WAVE. *What is a QR Code?* Available at: https://www.qrcode.com/en/about/ Accessed April 2026.

[2] DENSO WAVE. *QR Code Standardization*. Available at: https://www.qrcode.com/en/about/standards.html Accessed April 2026.

[3] Cybersecurity and Infrastructure Security Agency. *Phishing Guidance: Stopping the Attack Cycle at Phase One*. October 18, 2023. Available at: https://www.cisa.gov/resources-tools/resources/phishing-guidance-stopping-attack-cycle-phase-one

[4] National Institute of Standards and Technology. *Phishing*. Available at: https://www.nist.gov/node/1685311 Accessed April 2026.

[5] Android Developers. *Common Intents*. Available at: https://developer.android.com/guide/components/intents-common Accessed April 2026.

[6] Android Developers. *Common App Intents (API Level 30)*. Available at: https://developer.android.com/about/versions/11/reference/common-intents-30 Accessed April 2026.

[7] Heider A. M. Wahsheh and Flaminia L. Luccio. Security and Privacy of QR Code Applications: A Comprehensive Study, General Guidelines and Solutions. *Information*, 11(4):217, 2020. DOI: https://doi.org/10.3390/info11040217

[8] Marvin Kowalewski, Leona Lassak, Markus Dürmuth, and Theodor Schnitzler. Scanned and Scammed: Insecurity by ObsQRity? Measuring User Susceptibility and Awareness of QR Code-Based Attacks. In *Proceedings of the 34th USENIX Security Symposium*, 2025. Available at: https://www.usenix.org/conference/usenixsecurity25/presentation/kowalewski

[9] Xin Zhang, Xiaohan Zhang, Bo Zhao, Yuhong Nan, Zhichen Liu, Jianzhou Chen, Huijun Zhou, and Min Yang. Demystifying the (In)Security of QR Code-based Login in Real-world Deployments. In *Proceedings of the 34th USENIX Security Symposium*, 2025. Available at: https://www.usenix.org/system/files/usenixsecurity25-zhang-xin.pdf

[10] Health Sector Cybersecurity Coordination Center. *QR Code-Based Phishing (Quishing) as a Threat to the Health Sector*. October 23, 2023. Available at: https://www.aha.org/cybersecurity-government-intelligence-reports/2023-10-23-hc3-white-paper-tlp-clear-qr-code-based-phishing-quishing-threat

[11] United States Postal Inspection Service. *Quishing*. Last updated February 20, 2025. Available at: https://www.uspis.gov/news/scam-article/quishing

[12] Akinola, Boltwood, and Chaudhary. *QR-Code-Security-CMPT479*. GitHub repository, 2026. Available at: https://github.com/annieboltwood/QR-Code-Security-CMPT479